Assignment 2 – Source Code (Tasks 1–7)

Task 1 – Greedy / A* / UCS on grid maps (task1.py)

```
"""TASK 1 - Uniform-Cost, Greedy Best-First and A* on grid maps (campus shuttle)."""
import heapq, time, random, json
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

MOVES = [(1,0),(-1,0),(0,1),(0,-1)]          # 4-connected, unit step cost

def manhattan(a, b):
    """Admissible heuristic: exact cost in the RELAXED problem with no
obstacles."""      return abs(a[0]-b[0]) + abs(a[1]-b[1])

def neighbours(grid, p):
    r, c = p      for dr, dc in MOVES:          nr, nc = r+dr, c+dc          if 0 <= nr <
grid.shape[0] and 0 <= nc < grid.shape[1] and grid[nr, nc] == 0:              yield
(nr, nc)

def search(grid, start, goal, algo):
    """algo in {'UCS','GBFS','ASTAR'}. Priority: UCS=g, GBFS=h, A*=g+h. Tie-break: smaller h, then
FIFO."""
    t0 = time.perf_counter()     h = lambda p: manhattan(p, goal)     def
prio(g, p):          return g if algo == 'UCS' else (h(p) if algo == 'GBFS'
else g + h(p))     cnt = 0
    best_g = {start: 0}; parent = {start: None}
open_heap = [(prio(0, start), h(start), cnt, start)]
in_open = {start}; closed = set()     expanded = 0;
generated = 1; max_frontier = 1     while
open_heap:
        _, _, _, cur = heapq.heappop(open_heap)
        if cur in closed: continue                    # stale heap entry
        in_open.discard(cur)
if cur == goal:              path
= []; p = cur
            while p is not None: path.append(p); p = parent[p]
            return dict(found=True, cost=best_g[cur], expanded=expanded, generated=generated,
max_frontier=max_frontier, time_ms=(time.perf_counter()-t0)*1e3,
                        path=path[::-1], closed=closed)
        closed.add(cur); expanded += 1          for
nb in neighbours(grid, cur):              g2 =
```

```
best_g[cur] + 1            if nb in closed: continue
if nb not in best_g or g2 < best_g[nb]:
            # GBFS ignores g for ordering, but we still remember the first-found path
            if algo == 'GBFS' and nb in best_g: continue
            best_g[nb] = g2; parent[nb] = cur; cnt += 1; generated += 1
            heapq.heappush(open_heap, (prio(g2, nb), h(nb), cnt, nb)); in_open.add(nb)
        max_frontier = max(max_frontier, len(in_open))
    return dict(found=False, cost=None, expanded=expanded, generated=generated,
            max_frontier=max_frontier, time_ms=(time.perf_counter()-t0)*1e3, path=[], closed=closed)

# ---------------- maps (0 free, 1 obstacle) ----------------def
map_open(n=30):     g = np.zeros((n, n), int)     rng =
random.Random(1)
    for _ in range(6): g[rng.randrange(5, n-5), rng.randrange(5, n-5)] = 1   # a few isolated
closures     g[0, 0] = g[n-1, n-1] = 0     return g, (0, 0), (n-1, n-1)
def map_trap(n=30):
    """Dead-end trap / misleading-heuristic map: a long wall splits the map. Two gaps exist:     a gap
near the START (row 2, truly shortest) and a gap near the GOAL row (row 20, looks attractive to a
heuristic that only measures straight-line closeness). Greedy slides along the wall toward the goal
row,
    squeezing into the far gap, and returns a longer route."""
    g = np.zeros((n, n), int)
    g[:, 14] = 1                                # long wall     g[2, 14] =
0; g[20, 14] = 0                  # two gaps     return g, (5, 22),
(15, 4)

def map_edge(n=31):
    """Edge case: serpentine corridor maze (single winding path, all heuristics
misleading)."""     g = np.zeros((n, n), int)     for r in range(1, n-1, 2):         g[r, :] = 1
for i, r in enumerate(range(1, n-1, 2)):         g[r, n-2 if i % 2 == 0 else 1] = 0        #
alternate gaps => serpentine     return g, (0, 0), (n-1, n-1)

def map_random(n=30, density=0.30, seed=0):
    rng = np.random.default_rng(seed)
    g = (rng.random((n, n)) < density).astype(int); g[0, 0] = g[n-1, n-1] = 0
return g, (0, 0), (n-1, n-1)

def draw(ax, grid, start, goal, res, title):     img = np.zeros(grid.shape + (3,)); img[grid == 1] =
(0.15, 0.15, 0.15); img[grid == 0] = (1, 1, 1)
    for (r, c) in res['closed']: img[r, c] = (0.72, 0.85, 0.98)     for
(r, c) in res['path']: img[r, c] = (0.95, 0.55, 0.1)
    ax.imshow(img, interpolation='nearest'); ax.plot(start[1], start[0], 'g^', ms=7); ax.plot(goal[1], goal[0],
'r*', ms=9)
    ax.set_title(title, fontsize=8); ax.axis('off')
```

```
if __name__ == "__main__":     maps = {"Map A - Open road": map_open(), "Map B - Dead-end
trap": map_trap(), "Map C - Serpentine maze
(edge)": map_edge()}
   algos = [("UCS", "UCS"), ("Greedy", "GBFS"), ("A*",
"ASTAR")]     table = []; fig, axes = plt.subplots(3, 3,
figsize=(10, 10))     for i, (mname, (g, s, t)) in
enumerate(maps.items()):         opt = search(g, s, t,
'UCS')['cost']         for j, (label, a) in enumerate(algos):
r = search(g, s, t, a)
         row = dict(map=mname, algo=label, found=r['found'], cost=r['cost'], optimal=(r['cost'] == opt),
expanded=r['expanded'], generated=r['generated'], max_frontier=r['max_frontier'],
                  time_ms=round(r['time_ms'], 2))
         table.append(row)
         draw(axes[i][j], g, s, t, r, f"{mname}\n{label}: exp={r['expanded']}, cost={r['cost']}")
   plt.tight_layout(); plt.savefig("out/t1_maps.png", dpi=130); plt.close()

print(f"{'Map':32s} {'Algo':8s} {'Cost':>6s} {'Opt?':>6s} {'Expanded':>10s} {'Generated':>11s} {'MaxFront':
>10s}
{'ms':>8s}")     for r in table:
print(f"{r['map']:32s} {r['algo']:8s} {str(r['cost']):>6s} {str(r['optimal']):>6s} {r['expanded']:>10d}
{r['generated']:>11d} {r['max_frontier']:>10d} {r['time_ms']:>8.2f}")

   # extra: cluttered random maps, 30 seeds, several densities
   print("\nCluttered random 30x30 maps (30 seeds per density; solvable instances only)")
   print(f"{'density':>8s} {'algo':>8s} {'solved':>8s} {'avg exp':>9s} {'avg
cost':>10s} {'%optimal':>10s} {'avg subopt%':>13s}")     clutter = []     for dens in (0.10, 0.20, 0.25,
0.30):         agg = {a: [] for _, a in algos}; nsolv = 0         for sd in range(30):             g, s, t =
map_random(30, dens, sd)             ref = search(g, s, t, 'UCS')             if not ref['found']: continue
         nsolv += 1             for _, a in algos:                 r = search(g, s, t, a);
agg[a].append((r['expanded'], r['cost'], r['cost'] == ref['cost'], r['cost']/ref['cost']-1))         for label, a
in algos:             v = np.array(agg[a], float).reshape(-1, 4)
         print(f"{dens:8.2f} {label:>8s} {nsolv:8d} {v[:,0].mean():9.1f} {v[:,1].mean():10.1f}
{100*v[:,2].mean():10.1f} {100*v[:,3].mean():13.2f}")
         clutter.append(dict(density=dens, algo=label, solved=nsolv, avg_expanded=v[:,0].mean(),
avg_cost=v[:,1].mean(), pct_optimal=100*v[:,2].mean(), avg_subopt_pct=100*v[:,3].mean()))
   # chart
   fig, ax = plt.subplots(1, 2, figsize=(9, 3.6))
for label, _ in algos:         d = [c for c in clutter if
c['algo'] == label]
      ax[0].plot([c['density'] for c in d], [c['avg_expanded'] for c in d], 'o-', label=label)
ax[1].plot([c['density'] for c in d], [c['avg_subopt_pct'] for c in d], 'o-', label=label)
ax[0].set_xlabel("obstacle density"); ax[0].set_ylabel("avg nodes expanded"); ax[0].legend();
ax[0].set_title("Search effort")
   ax[1].set_xlabel("obstacle density"); ax[1].set_ylabel("avg path-cost excess (%)"); ax[1].set_title("Path
```

```
quality"); ax[1].legend()
    plt.tight_layout(); plt.savefig("out/t1_clutter.png", dpi=130); plt.close()
json.dump(dict(table=table, clutter=clutter), open("out/t1_raw.json", "w"), indent=1)
```

Task 2 – Online search agent (DFS, LRTA*) (task2.py)

```
"""TASK 2 - Online DFS agent and LRTA* in unmapped mazes (rover)."""
import random, json, collections
import numpy as np
import matplotlib;
matplotlib.use("Agg") import
matplotlib.pyplot as plt DIRS =
[(1,0),(0,1),(-1,0),(0,-1)]

def make_maze(size, braid=0.0, seed=0):
    """size x size cells (odd). Perfect maze by randomised DFS; then 'braid' = fraction of dead ends opened
    (lower dead-end density, creates loops). 1 = wall."""
rng = random.Random(seed); g = np.ones((size, size),
int)     def carve(r, c):          g[r, c] = 0          stack = [(r, c)]
while stack:              r, c = stack[-1]
          opts = [(r+2*dr, c+2*dc, dr, dc) for dr, dc in DIRS if 0 < r+2*dr < size-1 and 0 < c+2*dc < size-1
and g[r+2*dr, c+2*dc] == 1]
          if not opts: stack.pop(); continue
          nr, nc, dr, dc = rng.choice(opts); g[r+dr, c+dc] = 0; g[nr, nc] = 0; stack.append((nr, nc))      carve(1,
1)     if braid > 0:          deadends = [(r, c) for r in range(1, size-1) for c in range(1, size-1) if g[r, c] == 0
and sum(g[r+dr, c+dc] == 0 for dr, dc in DIRS) == 1]          rng.shuffle(deadends)          for (r, c) in
deadends[:int(braid*len(deadends))]:              walls = [(r+dr, c+dc) for dr, dc in DIRS if g[r+dr, c+dc] == 1
and 0 < r+2*dr < size-1 and 0 < c+2*dc
< size-1]
          if walls: wr, wc = rng.choice(walls); g[wr, wc] = 0
return g

def deadend_density(g):      free = [(r, c) for r in range(1, g.shape[0]-1) for c in range(1,
g.shape[1]-1) if g[r, c] == 0]
    de = [p for p in free if sum(g[p[0]+dr, p[1]+dc] == 0 for dr, dc in DIRS) == 1]
return len(de)/len(free)

def bfs_cost(g, s, t):      q =
collections.deque([(s, 0)]); seen = {s}
while q:          p, d = q.popleft()          if p ==
t: return d          for dr, dc in DIRS:              n
= (p[0]+dr, p[1]+dc)
          if g[n] == 0 and n not in seen: seen.add(n); q.append((n, d+1))
return None

def senses(g, p):              # the rover can only sense which ADJACENT cells are free
return [(p[0]+dr, p[1]+dc) for dr, dc in DIRS if g[p[0]+dr, p[1]+dc] == 0]

def online_dfs(g, start, target, max_moves=10**6):
    """Online DFS: move to an unvisited neighbour if any, else physically backtrack to the cell we came
from.
```

```
    Stops when target reached (success) or when back at the start with nothing unexplored (failure)."""
    pos = start; visited = {start}; stack = []; moves = 0; revisits = 0; known = {}      while moves
< max_moves:          if pos == target: return dict(found=True, moves=moves, revisits=revisits,
cells=len(visited))
        nbrs = senses(g, pos); known[pos] = nbrs
        nxt = next((n for n in nbrs if n not in visited), None)          if nxt is not None:
stack.append(pos); pos = nxt; visited.add(pos); moves += 1          elif stack:              pos = stack.pop();
moves += 1; revisits += 1          # backtrack = move into an already-visited cell          else:              return
dict(found=False, moves=moves, revisits=revisits, cells=len(visited))      return dict(found=False,
moves=moves, revisits=revisits, cells=len(visited)) def lrta_trials(g, start, target, trials=10,
max_moves=200000):
    """LRTA*: H table persists across trials. unit step cost;
H(unknown)=Manhattan."""      H = {}
    h0 = lambda p: abs(p[0]-target[0]) + abs(p[1]-target[1])
    H0 = lambda p: H.get(p, h0(p))      out = []
for _ in range(trials):          pos = start; moves =
0          while pos != target and moves <
max_moves:              nb = senses(g, pos)
          H[pos] = min(1 + H0(n) for n in nb)          # LRTA* update: H(s) <- min over neighbours of (cost
+ H(n))
          pos = min(nb, key=lambda n: 1 + H0(n)); moves
+= 1          out.append(moves)      return out

def farthest_cell(g, s, frac):
    """target at a given fraction of the maximum BFS distance from start."""
    dist = {s: 0}; q = collections.deque([s])
while q:          p = q.popleft()          for dr,
dc in DIRS:              n = (p[0]+dr,
p[1]+dc)
          if g[n] == 0 and n not in dist: dist[n] = dist[p]+1; q.append(n)
    mx = max(dist.values()); tgt = min(dist, key=lambda c: abs(dist[c]-frac*mx)); return tgt, dist[tgt]

if __name__ == "__main__":
    S = (1, 1); R = {}
    # ---- Exp A: target distance (31x31 perfect-ish maze, braid 0.2), 30 mazes ----
    print("EXP A: target distance (31x31 maze, braid=0.2, 30 mazes)")
    print(f"{'target dist':>12s}{'opt cost':>10s}{'moves':>9s}{'revisits':>10s}{'comp. ratio':>13s}{'worst
ratio':>13s}")      A = []      for frac in (0.1, 0.25, 0.5, 0.75, 1.0):
rows = []          for sd in range(30):              g = make_maze(31, 0.2,
sd); t, opt = farthest_cell(g, S, frac)
          if opt == 0: continue
          r = online_dfs(g, S, t); rows.append((opt, r['moves'], r['revisits'], r['moves']/opt))
        v = np.array(rows); A.append((frac, *v.mean(0), v[:, 3].max()))

print(f"{frac:12.2f}{v[:,0].mean():10.1f}{v[:,1].mean():9.1f}{v[:,2].mean():10.1f}{v[:,3].mean():13.2f}
```

```
{v[:,3].max():13.2f}")
   # ---- Exp B: maze size ----
   print("\nEXP B: maze size (far target = 75% of max distance, braid=0.2, 30 mazes)")
   print(f"{'size':>6s}{'free cells':>11s}{'opt cost':>10s}{'moves':>9s}{'revisits':>10s}{'comp. ratio':>13s}")
   B = []
   for size in (11, 21, 31, 41, 61):
       rows = []
       for sd in range(30):
           g = make_maze(size, 0.2, sd); t, opt = farthest_cell(g, S, 0.75)
           r = online_dfs(g, S, t); rows.append((int((g == 0).sum()), opt, r['moves'], r['revisits'], r['moves']/opt))
       v = np.array(rows); B.append((size, *v.mean(0)))

print(f"{size:6d}{v[:,0].mean():11.0f}{v[:,1].mean():10.1f}{v[:,2].mean():9.1f}{v[:,3].mean():10.1f}{v[:,4].mean():13.2f}")
   # ---- Exp C: dead-end density ----
   print("\nEXP C: dead-end density (31x31, target 75%, 30 mazes)")
print(f"{'braid':>6s}{'dead-end dens.':>15s}{'opt cost':>10s}{'moves':>9s}{'revisits':>10s}{'comp. ratio':>13s}")
   C = []
   for braid in (0.0, 0.25, 0.5, 0.75, 1.0):
       rows = []
       for sd in range(30):
           g = make_maze(31, braid, sd); t, opt = farthest_cell(g, S, 0.75)
           r = online_dfs(g, S, t); rows.append((deadend_density(g), opt, r['moves'], r['revisits'], r['moves']/opt))
       v = np.array(rows); C.append((braid, *v.mean(0)))

print(f"{braid:6.2f}{v[:,0].mean():15.3f}{v[:,1].mean():10.1f}{v[:,2].mean():9.1f}{v[:,3].mean():10.1f}{v[:,4].mean():13.2f}")
   # ---- Exp D: unreachable target ----
   print("\nEXP D: unreachable target (target cell sealed off; agent must exhaust the reachable region)")
   print(f"{'size':>6s}{'free cells':>11s}{'moves to give up':>18s}{'revisits':>10s}{'moves/cell':>12s}")
   D = []
   for size in (11, 21, 31, 41):
       rows = []
       for sd in range(30):
           g = make_maze(size, 0.2, sd)
           g[size-2, size-2] = 0; g[size-3, size-2] = 1; g[size-2, size-3] = 1   # sealed pocket
           free_reach = int((g == 0).sum()) - 1
           r = online_dfs(g, S, (size-2, size-2)); rows.append((free_reach, r['moves'], r['revisits'], r['moves']/free_reach, r['found']))
       v = np.array(rows, float); assert v[:, 4].sum() == 0
       D.append((size, *v.mean(0)[:4]))

print(f"{size:6d}{v[:,0].mean():11.0f}{v[:,1].mean():18.1f}{v[:,2].mean():10.1f}{v[:,3].mean():12.2f}")
   # ---- Exp E: LRTA* learning across trials ----
   print("\nEXP E: LRTA* over repeated missions (31x31, braid 0.2, target 75%, 20 mazes, 60 trials) - avg moves per trial")
   trials_m = []; opts = []; dfs_m = []
   for sd in range(20):
       g = make_maze(31, 0.2, sd); t, opt = farthest_cell(g, S, 0.75)
       trials_m.append(lrta_trials(g, S, t, 60)); opts.append(opt); dfs_m.append(online_dfs(g, S, t)['moves'])
```

```
    tm = np.array(trials_m).mean(0)
show = [0, 1, 2, 4, 9, 19, 29, 44, 59]
    print("trial:        " + "".join(f"{i+1:8d}" for i in show))
print("avg moves:    " + "".join(f"{tm[i]:8.1f}" for i in show))
    print(f"avg optimal cost = {np.mean(opts):.1f}; avg online-DFS moves (every mission) =
{np.mean(dfs_m):.1f}")     print("competitive ratio LRTA* at those trials: " + "
".join(f"{tm[i]/np.mean(opts):.2f}" for i in show))
    # ---- Changed condition: 20 rovers, 20 facilities, static maps ----
    odfs = np.mean(dfs_m); one_time_explore = np.mean([online_dfs(make_maze(31, .2, sd), S, (-1, -
1))['moves']
for sd in range(20)])
    print("\nCHANGED CONDITION: cumulative travel for N missions per facility (31x31 maze, avg of
20 mazes)")     print(f"{'N missions':>11s}{'online DFS every time':>24s}{'map once +
plan':>18s}{'LRTA* learning':>16s}")
    tot = []     for N in (1, 2, 5, 10, 50, 200):         dfs_total = N*odfs; map_once =
one_time_explore + (N-1)*np.mean(opts) + 0  # first run explores everything then returns, later
runs optimal         lr = sum(tm[:min(N, 60)]) + max(0, N-60)*tm[-1]
        tot.append((N, dfs_total, map_once, lr));
print(f"{N:11d}{dfs_total:24.0f}{map_once:18.0f}{lr:16.0f}")     print(f"(one full-exploration run costs
{one_time_explore:.0f} moves; x20 facilities x20 rovers multiplies
every column by 20)")
    json.dump(dict(A=A, B=B, C=C, D=D, E=tm.tolist(), opt=float(np.mean(opts)), dfs=float(odfs),
tot=tot,
explore=float(one_time_explore)), open("out/t2_raw.json", "w"), indent=1)
    # charts + sample maze
    fig, ax = plt.subplots(1, 4, figsize=(15, 3.4))
    ax[0].plot([a[0] for a in A], [a[4] for a in A], 'o-'); ax[0].set_xlabel("target distance (fraction of
max)"); ax[0].set_ylabel("competitive ratio"); ax[0].set_title("A: target distance")
ax[1].plot([b[0] for b in B], [b[5] for b in B], 'o-'); ax[1].set_xlabel("maze size");
ax[1].set_ylabel("competitive ratio"); ax[1].set_title("B: maze size")
    ax[2].plot([c[1] for c in C], [c[5] for c in C], 'o-'); ax[2].set_xlabel("dead-end density");
ax[2].set_ylabel("competitive ratio"); ax[2].set_title("C: dead-end density")
    ax[3].plot(range(1, 61), tm, 'o-', label="LRTA*"); ax[3].axhline(odfs, color='r', ls='--', label="online
DFS"); ax[3].axhline(np.mean(opts), color='g', ls=':', label="optimal (full map)")
    ax[3].set_xlabel("trial"); ax[3].set_ylabel("moves"); ax[3].set_title("E: learning across trials");
ax[3].legend()
    plt.tight_layout(); plt.savefig("out/t2_charts.png", dpi=130); plt.close()
    fig, ax = plt.subplots(1, 3, figsize=(10, 3.6))
    for a, (b, ttl) in zip(ax, [(0.0, "perfect maze (many dead ends)"), (0.5, "braid 0.5"), (1.0, "braid 1.0 (no
dead ends)")]):          g = make_maze(31, b, 3); a.imshow(g, cmap='gray_r'); a.set_title(ttl, fontsize=9);
a.axis('off')
    plt.tight_layout(); plt.savefig("out/t2_mazes.png", dpi=130); plt.close()
```

Task 3 – Alpha-beta pruning (task3.py)

```
"""TASK 3 - Alpha-beta behaviour vs game-tree structure."""
```

```
import random, json, math
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
INF = float('inf')

# A tree is either a number (leaf) or a list of children. def
random_tree(b, d, rng):
    return rng.randint(1, 10**6) if d == 0 else [random_tree(b, d-1, rng) for _ in range(b)]

def minimax(node, maxing, c):
c['nodes'] += 1
    if not isinstance(node, list): c['leaves'] += 1; return node
    vals = [minimax(ch, not maxing, c) for ch in node]
return max(vals) if maxing else min(vals)

def alphabeta(node, alpha, beta, maxing, c):
c['nodes'] += 1
    if not isinstance(node, list): c['leaves'] += 1; return node     if maxing:
v = -INF         for ch in node:             v = max(v, alphabeta(ch, alpha, beta,
False, c)); alpha = max(alpha, v)
            if alpha >= beta: c['cutoffs'] += 1; break         return v     v =
INF     for ch in node:         v = min(v, alphabeta(ch, alpha, beta, True,
c)); beta = min(beta, v)
        if alpha >= beta: c['cutoffs'] += 1; break
return v

def value(node, maxing):     if not
isinstance(node, list): return node
    vs = [value(ch, not maxing) for ch in node]; return max(vs) if maxing else min(vs)

def order(node, maxing, best_first=True):
    """Re-order children at every node: best-first (best for the player to move first) or worst-first."""
    if not isinstance(node, list): return node
    kids = [order(ch, not maxing, best_first) for ch in node]
    kids.sort(key=lambda k: value(k, not maxing), reverse=(maxing == best_first))
return kids

def shuffle_tree(node, rng):     if not
isinstance(node, list): return node
    kids = [shuffle_tree(k, rng) for k in node]; rng.shuffle(kids); return kids

def engineered_worst_tree(b, d):     """Leaf value = sum_k sign_k * i_k * 10^(d-k) where i_k is the
child index chosen at depth k.     sign = + where MAX moves (value rises with index) and - where
MIN moves (value falls with index):     every player's BEST move is its LAST child, and earlier
levels dominate later ones lexicographically."""     def build(k, acc):         if k == d: return acc
sign = 1 if k % 2 == 0 else -1
```

```
        return [build(k+1, acc + sign*i*10**(d-k-1)) for i in range(b)]
return build(0, 0)

def equal_tree(b, d, v=5):     return v if d == 0 else [equal_tree(b,
d-1, v) for _ in range(b)]

def run(tree):     m = dict(nodes=0, leaves=0, cutoffs=0); vm = minimax(tree,
True, m)     a = dict(nodes=0, leaves=0, cutoffs=0); va = alphabeta(tree, -INF,
INF, True, a)
    assert vm == va     return vm, m, a def best_case_leaves(b, d): return

b**math.ceil(d/2) + b**math.floor(d/2) - 1 if __name__ ==

"__main__":     b, d = 4, 6; rng = random.Random(42)     base =

random_tree(b, d, rng)

    trees = {"T1 best-ordered (many cutoffs)": order(base, True, True),
             "T2 engineered worst (no cutoffs)": engineered_worst_tree(b, d),
"T2b random tree sorted worst-first": order(base, True, False),
"T3a random order": base,
             "T3b all leaves equal": equal_tree(b, d)}
    print(f"b={b}, d={d}: minimax visits b^d = {b**d} leaves, {(b**(d+1)-1)//(b-1)} nodes in total")
    print(f"PREDICTIONS (written before running):")
    print(f"  T1 best-ordered : leaves ~ b^ceil(d/2)+b^floor(d/2)-1 = {best_case_leaves(b, d)}")
    print(f"  T2 engineered worst: leaves = b^d = {b**d} (zero cutoffs; best move is always LAST at every
node)")     print(f"  T2b merely sorted worst-first by local value: slightly below b^d (deep bounds from
ancestors still
cut a few branches)")
    print(f"  T3a random order: between the two, ~ b^(3d/4) = {b**(3*d/4):.0f} leaves on average")
print(f"  T3b equal leaves: ties trigger cutoffs (alpha>=beta) -> behaves like best-ordered:
~{best_case_leaves(b, d)}
leaves\n")     res = []
    print(f"{'Tree':40s}{'minimax leaves':>15s}{'AB leaves':>10s}{'AB
nodes':>10s}{'cutoffs':>9s}{'% leaves saved':>16s}")     for name, t in trees.items():         v, m, a =
run(t)
        print(f"{name:40s}{m['leaves']:15d}{a['leaves']:10d}{a['nodes']:10d}{a['cutoffs']:9d}{100*(1-
a['leaves']/m['leaves']):16.1f}")
        res.append(dict(tree=name, minimax_nodes=m['nodes'], minimax_leaves=m['leaves'],
ab_nodes=a['nodes'],
ab_leaves=a['leaves'], cutoffs=a['cutoffs']))
    # random order over many trees     ls = []     for sd in range(200):         r =
random.Random(sd); t = random_tree(b, d, r); _, _, a = run(t); ls.append(a['leaves'])
    print(f"\nT3a repeated on 200 random trees: mean AB leaves = {sum(ls)/len(ls):.0f} (min {min(ls)},
max
{max(ls)}); prediction ~{b**(3*d/4):.0f}")
```

```
    # depth sweep
    print("\nDepth sweep, b=4: alpha-beta leaves visited")
    print(f"{'d':>3s}{'minimax b^d':>13s}{'best-ordered':>14s}{'formula':>9s}{'random(avg of 20)':>19s}{'worst-ordered':>15s}{'equal leaves':>14s}")     sweep = []     for dd in range(2, 9):         base = random_tree(b, dd, random.Random(7)) if dd <= 7 else None         if dd <= 7:             best = run(order(base, True, True))[2]['leaves']; worst = run(engineered_worst_tree(b, dd))[2]['leaves']
        else:  # d=8: construct best/worst without sorting full tree (too slow) -> reuse formula only
            best = worst = None
        rnd = sum(run(random_tree(b, dd, random.Random(s)))[2]['leaves'] for s in range(20))/20 if dd <= 7 else None
        eq = run(equal_tree(b, dd))[2]['leaves']
        sweep.append((dd, b**dd, best, best_case_leaves(b, dd), rnd, worst, eq))
        f = lambda x: f"{x:.0f}" if x is not None else "-"
        print(f"{dd:3d}{b**dd:13d}{f(best):>14s}{best_case_leaves(b, dd):9d}{f(rnd):>19s}{f(worst):>15s}  {eq:14d}")
    # branching factor sweep at d=6, same b,d but different orders
    json.dump(dict(main=res, sweep=sweep), open("out/t3_raw.json", "w"), indent=1)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4))
    names = ["Best", "Random", "Worst", "Equal"]; vals = [res[0]['ab_leaves'], sum(ls)/len(ls), res[1]['ab_leaves'], res[4]['ab_leaves']]
    bars = ax[0].bar(names, vals, color=['#2a9d8f', '#e9c46a', '#e76f51', '#8ab17d']); ax[0].axhline(b**d, ls='--', color='k'); ax[0].text(0.1, b**d*1.02, f"minimax = {b**d}")
    for bb, v in zip(bars, vals): ax[0].text(bb.get_x()+bb.get_width()/2, v+40, f"{v:.0f}", ha='center')
ax[0].set_ylabel("leaf nodes visited"); ax[0].set_title(f"Same b={b}, d={d}: very different work")
    ds = [s[0] for s in sweep if s[2]]
    ax[1].semilogy(ds, [s[1] for s in sweep if s[2]], 'k--o', label="minimax b^d"); ax[1].semilogy(ds, [s[2] for
s in sweep if s[2]], 'o-', label="best-ordered")
    ax[1].semilogy(ds, [s[4] for s in sweep if s[2]], 'o-', label="random"); ax[1].semilogy(ds, [s[5] for s in sweep if s[2]], 'o-', label="worst-ordered")
    ax[1].set_xlabel("depth d"); ax[1].set_ylabel("leaves visited (log)"); ax[1].legend();
ax[1].set_title("Growth with depth (b=4)")
    plt.tight_layout(); plt.savefig("out/t3_charts.png", dpi=130); plt.close()
```

Task 4 – Depth-limited minimax (Connect-Four) (task4.py)

```
"""TASK 4 - Depth-limited plain minimax (no pruning) for Connect-Four on R x C boards."""
import random, time, json, math
import matplotlib;
matplotlib.use("Agg") import
matplotlib.pyplot as plt import
numpy as np

class C4:     def __init__(self, rows=6,
cols=7, k=4):         self.R, self.C, self.k =
rows, cols, k
        self.windows = []         for r in range(rows):
for c in range(cols):                 for dr, dc in
((0,1),(1,0),(1,1),(1,-1)):
                    er, ec = r+(k-1)*dr, c+(k-1)*dc                    if 0 <= er < rows
and 0 <= ec < cols:                         self.windows.append([(r+i*dr, c+i*dc)
for i in range(k)])
    def new(self): return [[0]*self.C for _ in range(self.R)]     def
moves(self, b): return [c for c in range(self.C) if b[0][c] == 0]     def
play(self, b, c, p):          for r in range(self.R-1, -1, -1):              if
b[r][c] == 0: b[r][c] = p; return r     def undo(self, b, c):          for r in
range(self.R):              if b[r][c] != 0: b[r][c] = 0; return     def
winner(self, b):          for w in self.windows:
            s = b[w[0][0]][w[0][1]]
            if s != 0 and all(b[r][c] == s for r, c in w): return s
return 0     def evaluate(self, b):
        """Own evaluation function (from player 1's view): sum over every length-k window containing
only one          colour: 1 / 10 / 100 points for 1 / 2 / 3 discs (k=4), +3 per disc in the centre
column(s)."""          sc = 0          for w in self.windows:              a = sum(1 for r, c in w if b[r][c] == 1); o
= sum(1 for r, c in w if b[r][c] == 2)
            if a and not o: sc += 10**(a-1)              elif o
and not a: sc -= 10**(o-1)          mid = self.C//2
for r in range(self.R):              sc += 3*(b[r][mid] ==
1) - 3*(b[r][mid] == 2)          return sc

def minimax(g, b, depth, maxing, st):
    """maxing: player 1 to move. Counts nodes (calls) and evaluation calls."""
    st['nodes'] += 1
w = g.winner(b)
    if w: st['evals'] += 1; return (10**6 + depth) if w == 1 else -(10**6 + depth)
    mv = g.moves(b)
    if not mv: st['evals'] += 1; return 0
    if depth == 0: st['evals'] += 1; return g.evaluate(b)              # cut-off: heuristic evaluation
    best = -math.inf if maxing else math.inf
for c in mv:
```

```
        g.play(b, c, 1 if maxing else 2)         v =
minimax(g, b, depth-1, not maxing, st)
        g.undo(b, c)
        best = max(best, v) if maxing else min(best, v)
return best

def choose(g, b, depth, me,
st):     best, bc = None, None
for c in g.moves(b):
        g.play(b, c, me)
        v = minimax(g, b, depth-1, me == 2, st) if me == 1 else minimax(g, b, depth-1, True, st)
g.undo(b, c)
        score = v if me == 1 else -v
        if best is None or score > best: best, bc = score, c
    return bc

def play_game(g, depth, ai_first, rng, st, opp_depth=0):
    """AI (given depth) vs random player (opp_depth=0) or vs a shallower minimax player. Returns (result,
AI moves)."""
    b = g.new(); turn = 1; ai = 1 if ai_first else 2; n_ai = 0; dummy = dict(nodes=0,
evals=0)     while True:          mv = g.moves(b)          if not mv: return 0, n_ai
        if turn == ai: c = choose(g, b, depth, ai, st); n_ai += 1
elif opp_depth: c = choose(g, b, opp_depth, turn, dummy)
        else: c = rng.choice(mv)
        g.play(b, c, turn)
        if g.winner(b): return (1 if turn == ai else -1), n_ai
turn = 3 - turn

if __name__ ==
"__main__":     out = {}
    # ----- (1) node-count derivation check on the empty 6x7 board -----
    g = C4(6, 7); print("Theory: nodes N(b,d) = (b^(d+1)-1)/(b-1); leaves(terminal evaluations) = b^d
(valid
while no win/full-board occurs)")
    print(f"{'depth d':>8s}{'theory nodes':>14s}{'measured nodes':>16s}{'theory leaves':>15s}{'measured
evals':>16s}{'time (s)':>10s}")
    rows = []     for d in range(1, 8):          st = dict(nodes=0, evals=0); b = g.new(); t0 =
time.perf_counter(); minimax(g, b, d, True, st); dt = time.perf_counter()-t0
        th = (7**(d+1)-1)//6; rows.append((d, th, st['nodes'], 7**d, st['evals'], dt))
print(f"{d:8d}{th:14d}{st['nodes']:16d}{7**d:15d}{st['evals']:16d}{dt:10.3f}")
out['growth'] = rows
    # ----- (2) different board widths (branching factor) at fixed depth 5 -----
    print("\nBoard size effect (rows=6, depth=5, empty board)")
    print(f"{'board':>8s}{'b':>4s}{'theory nodes':>14s}{'measured':>10s}{'evals':>10s}{'time (s)':>10s}")
```

```
    rows2 = []     for cols in (4, 5, 6, 7, 8):         gg = C4(6, cols); st = dict(nodes=0, evals=0); t0 = time.perf_counter(); minimax(gg, gg.new(), 5, True,
st); dt = time.perf_counter()-t0
        th = (cols**6-1)//(cols-1); rows2.append((cols, th, st['nodes'], st['evals'], dt)); print(f"6x{cols:<6d}{cols:4d}{th:14d}{st['nodes']:10d}{st['evals']:10d}{dt:10.3f}")
out['cols'] = rows2
    # ----- (3) playing strength: 20 games vs random for depths 1..4 (10 as first, 10 as second) -----
    print("\nPlaying strength vs RANDOM player (6x7, 20 games: 10 AI-first, 10 AI-second)")
print(f"{'depth':>6s}{'wins':>6s}{'draws':>7s}{'losses':>8s}{'win rate':>10s}{'AI moves':>10s}{'nodes/move':>12s}{'evals/move':>12s}{'s/move':>9s}")
    rows3 = []     for d in
(1, 2, 3, 4):
        rng = random.Random(100+d); res = []; st = dict(nodes=0, evals=0); nm = 0; t0 = time.perf_counter()         for gi in range(20):             r, n = play_game(g, d, gi % 2 == 0, rng, st); res.append(r); nm += n         dt = time.perf_counter()-t0; w, dr, l = res.count(1), res.count(0), res.count(-1)         rows3.append((d, w, dr, l, w/20, nm, st['nodes']/nm, st['evals']/nm, dt/nm))

print(f"{d:6d}{w:6d}{dr:7d}{l:8d}{w/20:10.2f}{nm:10d}{st['nodes']/nm:12.0f}{st['evals']/nm:12.0f}{dt / nm:9.3f}")
    out['strength'] = rows3
    # random is a weak yardstick (all depths win ~100%) -> also play deeper AI vs shallower AI (depth 1), 20 games
    print("\nPlaying strength vs a DEPTH-1 minimax opponent (20 games each, deeper AI alternates first/second)")
    print(f"{'AI depth':>9s}{'wins':>6s}{'draws':>7s}{'losses':>8s}{'win rate':>10s}")
    rows4 = []     for d in (1, 2, 3):         rng = random.Random(7); res = []         for gi in range(20):
r, _ = play_game(g, d, gi % 2 == 0, rng, dict(nodes=0, evals=0), opp_depth=1); res.append(r)
        rows4.append((d, res.count(1), res.count(0), res.count(-1), res.count(1)/20))
        print(f"{d:9d}{res.count(1):6d}{res.count(0):7d}{res.count(-1):8d}{res.count(1)/20:10.2f}")
out['vs_d1'] = rows4
    # ----- practical limit: time per move on the empty board and mid-game board -----
    json.dump(out, open("out/t4_raw.json", "w"), indent=1)
    d = [r[0] for r in out['growth']]
    fig, ax = plt.subplots(1, 3, figsize=(14, 3.8))
    ax[0].semilogy(d, [r[1] for r in out['growth']], 'k--', label="theory (b^(d+1)-1)/(b-1)"); ax[0].semilogy(d, [r[2] for r in out['growth']], 'o', label="measured nodes")
    ax[0].semilogy(d, [r[4] for r in out['growth']], 's', label="evaluation calls"); ax[0].set_xlabel("depth d"); ax[0].set_ylabel("count (log)"); ax[0].legend(); ax[0].set_title("6x7: theory vs measured")
    ax[1].semilogy(d, [r[5] for r in out['growth']], 'o-', color='C3'); ax[1].axhline(1.0, ls='--', color='k'); ax[1].text(1, 1.3, "1 s/move limit"); ax[1].set_xlabel("depth d"); ax[1].set_ylabel("seconds (log)"); ax[1].set_title("Time per move (empty 6x7)")
    x = np.arange(4); ax[2].bar(x-0.2, [r[4] for r in out['strength']], 0.4, label='vs random', color='#2a9d8f'); ax[2].bar(x[:3]+0.2, [r[4] for r in out['vs_d1']], 0.4, label='vs depth-1 AI', color='#e9c46a'); ax[2].set_xticks(x); ax[2].set_xticklabels([f'd={i}' for i in range(1,5)]); ax[2].set_ylim(0, 1.05);
```

```
ax[2].set_ylabel('win rate (20 games)'); ax[2].legend(); ax[2].set_title('Move quality')
    plt.tight_layout(); plt.savefig("out/t4_charts.png", dpi=130); plt.close()
```

Task 5 – Constraint propagation (AC-3) for timetabling (task5.py)

```
"""TASK 5 - Exam timetabling: AC-3 style constraint propagation derived from the support principle + backtracking."""
import random, json, time, collections, sys
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
sys.setrecursionlimit(10000)

class CSP:
    """Variables = exams; domains = sets of slots; constraints = for every pair of exams sharing students, the binary relation 'slot(x) != slot(y)'. Constraints are stored as an adjacency map: nbrs[x] = set of y."""
def __init__(self, domains, conflicts):        self.vars = list(domains); self.dom0 = {v: set(d) for v, d in domains.items()}
        self.nbrs = {v: set() for v in domains}
        for a, b in conflicts: self.nbrs[a].add(b); self.nbrs[b].add(a)
self.checks = 0
    def compatible(self, vx, x, vy, y):       # ONE constraint check
self.checks += 1        return x != y

def revise(csp, dom, xi, xj, log=None):
    """Remove every value of xi that has NO supporting value in xj (the given principle)."""
    removed = False     for x in list(dom[xi]):         if not any(csp.compatible(xi, x, xj, y) for y in dom[xj]):    # unsupported?             dom[xi].discard(x); removed = True             if log is not None: log.append((xi, x, xj))     return removed

def ac3(csp, dom, queue=None, log=None, stats=None):
    """Queue holds directed arcs (xi, xj) 'xi must be supported by xj'. When D[xi] shrinks, every arc (xk, xi)
    (k a neighbour of xi, k != xj) may have lost its support, so it is re-queued. Returns False on wipe-out."""
    if queue is None: queue = collections.deque((xi, xj) for xi in csp.vars for xj in csp.nbrs[xi])
    queue = collections.deque(queue); inq = set(queue)
while queue:        xi, xj = queue.popleft(); inq.discard((xi, xj))        if revise(csp, dom, xi, xj, log):
if not dom[xi]:                if stats is not None: stats['wipeouts'] += 1
                return False                                    # domain wipe-out => no solution below here            for xk in csp.nbrs[xi]:                if xk != xj and (xk, xi) not in inq: queue.append((xk, xi)); inq.add((xk, xi))
return True

def backtrack(csp, dom, assign, stats, use_prop, node_cap):
    """Static variable order (so only propagation differs between the compared methods)."""
    if stats['nodes'] >= node_cap: stats['capped'] = True; return None     if len(assign) == len(csp.vars): return dict(assign)     v =
```

```
next(x for x in csp.vars if x not in assign)     for val in
sorted(dom[v]):         stats['nodes'] += 1
       if any(not csp.compatible(v, val, u, assign[u]) for u in csp.nbrs[v] if u in assign): continue   #
consistency check
       new = {k: set(s) for k, s in dom.items()}; new[v] = {val}; assign[v] = val
ok = True
       if use_prop: ok = ac3(csp, new, queue=[(u, v) for u in csp.nbrs[v] if u not in assign], stats=stats)   #
MAC         if ok:             r = backtrack(csp, new, assign, stats, use_prop, node_cap)
          if r is not None: return r
          if stats.get('capped'): del assign[v]; return None
       del assign[v]; stats['backtracks'] += 1
return None

def solve(csp, use_prop, node_cap=30000):     csp.checks = 0; stats =
dict(nodes=0, backtracks=0, wipeouts=0, capped=False)     dom = {v: set(d)
for v, d in csp.dom0.items()}     if use_prop:         if not ac3(csp, dom,
stats=stats): return None, stats, csp.checks, dom
   sol = backtrack(csp, dom, {}, stats, use_prop, node_cap)
return sol, stats, csp.checks, dom

def draw_domains(ax, csp, dom, nslots,
title):     M = np.zeros((len(csp.vars),
nslots))     for i, v in enumerate(csp.vars):
for s in dom[v]: M[i, s-1] = 1
   ax.imshow(M, cmap='Blues', vmin=0, vmax=1.3, aspect='auto'); ax.set_title(title, fontsize=9)
ax.set_xticks(range(nslots)); ax.set_xticklabels([f"S{s}" for s in range(1, nslots+1)], fontsize=7);
ax.set_yticks(range(len(csp.vars))); ax.set_yticklabels(csp.vars, fontsize=7)     for i in
range(M.shape[0]):         for j in range(M.shape[1]): ax.text(j, i, "x" if M[i, j] else "", ha='center',
va='center', color='w', fontsize=8)

def random_instance(n, p, k, seed,
domain_spread=False):     rng = random.Random(seed)
   edges = [(f"E{i}", f"E{j}") for i in range(n) for j in range(i+1, n) if rng.random() < p]
return CSP({f"E{i}": set(range(1, k+1)) for i in range(n)}, edges)

if __name__ == "__main__":     print("""PSEUDOCODE (student-derived from the support
principle)   revise(xi, xj): for each x in D[xi]: if no y in D[xj] with compatible(x,y): remove x;
return removed?   AC(CSP): Q = all directed arcs (xi,xj) for each conflicting pair; while Q not
empty: (xi,xj)=pop(Q)
                    if revise(xi,xj): if D[xi] empty -> return FAIL (domain wipe-out = no
   solution)                              else push (xk,xi) for every neighbour xk of xi with xk
                                                                                    != xj
         return OK
  SOLVE: if AC fails -> no solution; if every |D|=1 -> solution; else pick variable, try each value v:
copy domains, set D[x]={v}, run AC with Q = {(xk,x)}, recurse if no wipe-out, else undo (backtrack)\
n""")
```

```
    # ---------- Instance 1: propagation alone solves it ----------
    inst1 = CSP({"Maths": {1}, "Physics": {1, 2}, "Chem": {1, 2, 3}, "Bio": {2, 3}, "CS": {1, 2, 3}},
[("Maths", "Physics"), ("Physics", "Chem"), ("Chem", "Bio"), ("Bio", "CS"), ("Chem", "CS"),
("Maths", "Chem")])
    log = []; dom = {v: set(d) for v, d in inst1.dom0.items()}; snaps = [("initial domains", {v: set(d) for v, d
in dom.items()})]
    inst1.checks = 0; stats = dict(wipeouts=0); ok = True
    # re-run AC3 step by step to record snapshots after each removal
    q = collections.deque((a, b) for a in inst1.vars for b in inst1.nbrs[a]); inq = set(q); sizes = [sum(len(d)
for d in dom.values())]     while q:         xi, xj = q.popleft(); inq.discard((xi, xj)); l = []         if
revise(inst1, dom, xi, xj, l):             for (v, x, w) in l: print(f"  AC removes {x} from D[{v}] (no support
in {w})")             snaps.append((f"after revise({xi},{xj})", {v: set(d) for v, d in dom.items()}))
          sizes.append(sum(len(d) for d in dom.values()))             for xk in
inst1.nbrs[xi]:                 if xk != xj and (xk, xi) not in inq: q.append((xk, xi));
inq.add((xk, xi))
    print("Instance 1 final domains:", {v: sorted(d) for v, d in dom.items()}, "| checks =", inst1.checks, "|
total domain size trace:", sizes)     sol1,
st1, ck1, _ = solve(inst1, True)
    print("Instance 1: all domains became singletons -> solution read off with search nodes =",
st1["nodes"], sol1)
    pick = [0, 1, min(2, len(snaps)-1), len(snaps)-1]; fig, ax = plt.subplots(1, 4, figsize=(13, 3.2))
    for a, i in zip(ax, pick): draw_domains(a, inst1, snaps[i][1], 3, snaps[i][0])
plt.tight_layout(); plt.savefig("out/t5_inst1_domains.png", dpi=130); plt.close()
    # failure detected without search
    inst1b = CSP({"A": {1}, "B": {1, 2}, "C": {1, 2}}, [("A", "B"), ("A", "C"), ("B", "C")])
    solb, stb, ckb, domb = solve(inst1b, True)
    print("Instance 1b (triangle, A fixed to slot 1, B,C in {1,2}): solution =", solb, "| wipe-outs =",
stb['wipeouts'], "| search nodes =", stb['nodes'], "-> failure detected by propagation alone")
    solb2, stb2, ckb2, _ = solve(inst1b, False)
    print("    plain backtracking on the same instance: nodes =", stb2['nodes'], "backtracks =",
stb2['backtracks'], "checks =", ckb2)
    # ---------- Instance 2: designed to challenge ---------     # 3-colouring-like: 'not-equal' with full
domains => arc consistency removes nothing, search is needed.     # Hard core: K4 minus nothing is
infeasible with 3 slots but AC3 cannot see it (every value has support).     k4 = CSP({f"X{i}": {1, 2,
3} for i in range(1, 5)}, [(f"X{i}", f"X{j}") for i in range(1, 5) for j in range(i+1, 5)])
    s, st, ck, d2 = solve(k4, True)
    print("\nInstance 2 (K4 with 3 slots: 4 mutually clashing exams): AC alone reduced nothing:",
all(len(d2[v]) == 3 for v in d2), "| solution:", s,
          "| search nodes with MAC =", st['nodes'], "wipe-outs =", st['wipeouts'])
    # scaling test + comparison
    print("\nSCALING: random conflict graphs, k=5 slots, edge prob p=0.30 (near the hard region), 10
instances per n; node cap 30000")
    print(f"{'n':>4s}{'method':>9s}{'avg nodes':>12s}{'avg backtracks':>16s}{'avg checks':>13s}{'avg
```

```
wipeouts':>14s}{'capped':>8s}{'%solved':>9s}")     summary = []     for n in (10,
15, 20, 25, 30, 40):         for name, prop in (("BT", False), ("BT+AC3", True)):
agg = []; cap = 0; sol = 0             for sd in range(10):                 c =
random_instance(n, 0.30, 5, 1000*n+sd); s, st, ck, _ = solve(c, prop)
             agg.append((st['nodes'], st['backtracks'], ck, st['wipeouts'])); cap += st['capped']; sol += (s is not
None)
          v = np.array(agg, float).mean(0); summary.append((n, name, *v, cap, 100*sol/10))

print(f"{n:4d}{name:>9s}{v[0]:12.0f}{v[1]:16.0f}{v[2]:13.0f}{v[3]:14.1f}{cap:8d}{100*sol/10:9.0f}")
   json.dump(dict(summary=summary), open("out/t5_raw.json", "w"), indent=1)
   # ---------- visualise shrinking domains inside a search (MAC) on a random 12-exam instance ----------
   c = random_instance(12, 0.35, 4, 5); dom = {v: set(d) for v, d in c.dom0.items()}
   fig, ax = plt.subplots(1, 4, figsize=(13, 4.2)); draw_domains(ax[0], c, dom, 4, "start (full domains)")
   ac3(c, dom); draw_domains(ax[1], c, dom, 4, "after initial AC")     ordv = c.vars[:2]; d2 =
{v: set(s) for v, s in dom.items()}     for i, v in enumerate(ordv):         val = min(d2[v]); d2[v] =
{val}; ok = ac3(c, d2, queue=[(u, v) for u in c.nbrs[v]])         draw_domains(ax[2+i], c, d2, 4,
f"assign {v}={val}, then AC" + ("" if ok else " (WIPE-OUT)"))
   plt.tight_layout(); plt.savefig("out/t5_search_domains.png", dpi=130); plt.close()
   # scaling charts
   fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
for name in ("BT", "BT+AC3"):         r = [s for
s in summary if s[1] == name]
      ax[0].semilogy([x[0] for x in r], [max(x[2], 1) for x in r], 'o-', label=name); ax[1].semilogy([x[0] for
x in r], [max(x[4], 1) for x in r], 'o-', label=name)
   ax[0].set_xlabel("number of exams n"); ax[0].set_ylabel("avg search nodes (log)"); ax[0].legend();
ax[0].set_title("Nodes visited")
   ax[1].set_xlabel("number of exams n"); ax[1].set_ylabel("avg constraint checks (log)"); ax[1].legend();
ax[1].set_title("Constraint checks")
   plt.tight_layout(); plt.savefig("out/t5_scaling.png", dpi=130); plt.close()
```

Task 6 – Exhaustive vs local search (delivery tour) (task6.py)

```
"""TASK 6 - Delivery tour: exhaustive enumeration vs 2-opt local search with random
restarts.""" import math, random, time, json, itertools import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

def make_points(n, seed):     rng = random.Random(seed); return [(rng.uniform(0, 100),
rng.uniform(0, 100)) for _ in range(n)]
def dist_matrix(P): return [[math.dist(a, b) for b in P] for a in P]
def tour_len(D, tour):         # tour = [0, c1, ..., c_{n-1}] ; return to depot 0
return sum(D[tour[i]][tour[(i+1) % len(tour)]] for i in range(len(tour)))

# ---------- (A) exhaustive enumeration, designed from scratch ---------
def exhaustive(D):
   """Representation: a tour is a permutation list starting at depot 0 (depot fixed => (n-1)! tours).
Generation: depth-first construction of permutations with a 'used' mask (choose next unvisited
```

```
customer).  Evaluation: the cost is accumulated incrementally while extending the prefix; the closing edge is added at the leaf.
  Retention: keep (best_cost, best_tour) - replace only when a COMPLETE tour is strictly shorter."""  n = len(D); best = [math.inf, None]; count = [0]; path = [0]; used = [False]*n; used[0] = True  def extend(cost):  if len(path) == n:  total = cost + D[path[-1]][0]; count[0] += 1  if total < best[0]: best[0] = total; best[1] = path[:]  return  last = path[-1]  for c in range(1, n):  if not used[c]:  used[c] = True; path.append(c); extend(cost + D[last][c]); path.pop(); used[c] = False  extend(0.0); return best[0], best[1], count[0]

# ---------- (B) local search: 2-opt hill climbing + random restarts ---------
def two_opt_climb(D, tour):  n = len(tour); improved = True  while improved:
  improved = False  for i in range(1, n-1):  for j in range(i+1, n):  a, b = tour[i-1], tour[i]; c, d = tour[j], tour[(j+1) % n]
  delta = D[a][c] + D[b][d] - D[a][b] - D[c][d]  # O(1) move evaluation (reverse segment i..j)  if delta < -1e-12:  tour[i:j+1] = reversed(tour[i:j+1]); improved = True
return tour

def local_search(D, restarts, rng):  n = len(D); best = math.inf; bt = None  for _ in range(restarts):  t = [0] + rng.sample(range(1, n), n-1); t = two_opt_climb(D, t); L = tour_len(D, t)
  if L < best: best, bt = L, t[:]  return best, bt

if __name__ == "__main__":  out = {}
  # ---------- correctness ----------
  print("CORRECTNESS 1: depot + 4 customers at hand-checkable coordinates")  P = [(0, 0), (0, 3), (4, 3), (4, 0), (2, 1)]; D = dist_matrix(P)
  allt = sorted(((tour_len(D, [0]+list(p)), [0]+list(p)) for p in itertools.permutations(range(1, 5))))
print(f"  independent check via itertools.permutations: {len(allt)} tours; best = {allt[0][0]:.4f} {allt[0][1]}; worst = {allt[-1][0]:.4f}")
  c, t, k = exhaustive(D); print(f"  my exhaustive search : tours enumerated = {k}, best = {c:.4f} {t}  -> matches: {abs(c-allt[0][0]) < 1e-9}")
  print("CORRECTNESS 2: n=10 points on a circle (radius 50): optimum must be the perimeter 2*n*R*sin(pi/n)
(visiting in circular order)")
  n = 10; Pc = [(50*math.cos(2*math.pi*i/n), 50*math.sin(2*math.pi*i/n)) for i in (0, 3, 7, 1, 9, 5, 2, 8, 4, 6)]  # shuffled labelling
  Dc = dist_matrix(Pc); c, t, k = exhaustive(Dc); th = 2*n*50*math.sin(math.pi/n)
  print(f"  exhaustive = {c:.6f}, analytic perimeter = {th:.6f}, match = {abs(c-th) < 1e-9}, tours enumerated =
```

```
{k} (= 9! =
{math.factorial(9)})")     # -------
--- growth ----------
   print("\nGROWTH: tours enumerated (prediction (n-1)!) vs measured, random instances")
   print(f"{'n':>3s}{'predicted (n-1)!':>18s}{'measured':>12s}{'time (s)':>11s}{'time ratio to prev':>20s}
{'predicted ratio (n-
1)':>22s}")     growth = [];
prev = None     for n in
range(5, 12):
       D = dist_matrix(make_points(n, 11)); t0 = time.perf_counter(); c, t, k = exhaustive(D); dt =
time.perf_counter()-t0
       growth.append((n, math.factorial(n-1), k, dt, c))
       print(f"{n:3d}{math.factorial(n-1):18d}{k:12d}{dt:11.3f}{(dt/prev if prev else
float('nan')):20.2f}{(n-1
if n>5 else float('nan')):22.0f}"); prev = dt
   out['growth'] = growth
   # ---------- local search quality ----------
   print("\nLOCAL SEARCH vs exhaustive optimum: 20 runs each (independent random seeds), 2-opt
hill climbing")     print(f"{'n':>3s}{'restarts':>9s}{'mean gap %':>12s}{'max gap %':>11s}{'runs hitting
opt':>18s}{'mean time
(ms)':>16s}")     ls = []
for n in (8, 9, 10, 11):
       D = dist_matrix(make_points(n, 11)); opt =
exhaustive(D)[0]         for restarts in (1, 5, 20):
           gaps = []; tms = []             for run in range(20):                 rng = random.Random(1000*n +
10*restarts + run); t0 = time.perf_counter(); L, _ = local_search(D, restarts, rng);
tms.append((time.perf_counter()-t0)*1e3); gaps.append(100*(L-opt)/opt)
           hit = sum(g < 1e-9 for g in gaps); ls.append((n, restarts, np.mean(gaps), max(gaps), hit,
np.mean(tms)))
           print(f"{n:3d}{restarts:9d}{max(np.mean(gaps),0):12.2f}{max(max(gaps),0):11.2f}{hit:15d}/
20{np.mean(tms):16.2f}")
out['ls'] = ls
   # larger sizes (no exhaustive optimum): compare to best of 200 restarts
   print("\nLarger sizes (optimum unknown): gap of 5-restart 2-opt relative to best tour found in 200
restarts,
20 runs")     big = []     for n in
(15, 20, 30, 50, 100):
       D = dist_matrix(make_points(n, 3)); ref = local_search(D, 200, random.Random(5))[0]; gaps = [];
tms = []         for run in range(20):             t0 = time.perf_counter(); L, _ = local_search(D, 5,
random.Random(run));
tms.append((time.perf_counter()-t0)*1e3); gaps.append(100*(L-ref)/ref)
       big.append((n, np.mean(gaps), max(gaps), np.mean(tms))); print(f"  n={n:3d}   mean gap
{np.mean(gaps):5.2f}%  max {max(gaps):5.2f}%  mean time {np.mean(tms):8.1f} ms")
   out['big'] = big
```

```
    # extrapolate exhaustive time
per_tour = growth[-1][3]/growth[-1][2]
    print(f"\nExhaustive cost per complete tour ~ {per_tour*1e6:.2f} microseconds (measured at n=11)")
    print(f"{'n':>3s}{'tours':>20s}{'est. time':>16s}")
for n in (10, 11, 12, 13, 14, 15, 20):
        T = math.factorial(n-1)*per_tour; txt = f"{T:.2f} s" if T < 3600 else (f"{T/3600:.1f} h" if T <
86400*365 else f"{T/86400/365:.2e} years")
        print(f"{n:3d}{math.factorial(n-1):20d}{txt:>16s}")
json.dump(out, open("out/t6_raw.json", "w"), indent=1)
fig, ax = plt.subplots(1, 3, figsize=(14, 3.8))
    ax[0].semilogy([g[0] for g in growth], [g[1] for g in growth], 'k--', label="(n-1)! predicted");
ax[0].semilogy([g[0] for g in growth], [g[2] for g in growth], 'o', label="measured");
ax[0].set_xlabel("n");
ax[0].set_ylabel("tours enumerated"); ax[0].legend(); ax[0].set_title("Combinatorial growth")
    ax[1].semilogy([g[0] for g in growth], [g[3] for g in growth], 'o-', color='C3'); ax[1].axhline(2, ls='--',
color='k'); ax[1].text(5, 2.3, "2 s response limit"); ax[1].set_xlabel("n"); ax[1].set_ylabel("seconds (log)");
ax[1].set_title("Exhaustive search time")     for r in (1, 5, 20):         rr = [x for x in ls if x[1] == r];
ax[2].plot([x[0] for x in rr], [x[2] for x in rr], 'o-', label=f"{r} restart(s)")
    ax[2].set_xlabel("n"); ax[2].set_ylabel("mean gap to optimum (%)"); ax[2].legend(); ax[2].set_title("2-
opt solution quality")
    plt.tight_layout(); plt.savefig("out/t6_charts.png", dpi=130); plt.close()
    # tour picture: local optimum vs optimum
    P = make_points(11, 11); D = dist_matrix(P); optc, optt, _ = exhaustive(D); rng = random.Random(3)
    worst = None     for s in range(200):         t = two_opt_climb(D,
[0]+random.Random(s).sample(range(1, 11), 10))         L =
tour_len(D, t)
        if worst is None or L > worst[0]: worst = (L, t)
fig, ax = plt.subplots(1, 2, figsize=(9, 4.2))
    for a, (L, t, ttl) in zip(ax, [(optc, optt, "Exhaustive optimum"), (worst[0], worst[1], "Worst 2-opt local
minimum (1 start)")]):         xs = [P[i][0] for i in t+[t[0]]]; ys = [P[i][1] for i in t+[t[0]]]; a.plot(xs, ys, 'o-
'); a.plot(*P[0],
'rs', ms=9); a.set_title(f"{ttl}\nlength = {L:.1f}", fontsize=9)
    plt.tight_layout(); plt.savefig("out/t6_tours.png", dpi=130); plt.close()
    print(f"\nExample trapped run: local minimum {worst[0]:.2f} vs optimum {optc:.2f}
(+{100*(worst[0]/optc1):.1f}%)")
```

Task 7 – Redesign: branch-and-bound tour search (task7.py)

```
"""TASK 7 - Redesign of Task 6's exhaustive tour search: bound-driven search that reuses information.
Bottleneck (from Task 6): every one of the (n-1)! tours is built to the end, even when a partial tour is
already hopeless.  Redesign = keep the SAME search space and the SAME correctness (always returns
the true optimum) but
  (1) prune a prefix as soon as an O(1) optimistic estimate of its best completion cannot beat the incumbent,
  (2) visit promising children first so that a good incumbent appears early,
  (3) seed the incumbent with the local-search tour from Task 6 (reuse of earlier computation)."""
import math, random, time, json
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from task6 import make_points, dist_matrix, exhaustive, local_search, tour_len

def original_nodes(n):
    """number of partial tours extended by plain exhaustive search = sum_{k=0}^{n-1} (n-1)!/(n-1-k)!"""
    return sum(math.factorial(n-1)//math.factorial(n-1-k) for k in range(n))

def bnb(D, use_lb=True, nearest_first=True, seed_ub=True, rng=None):
    n = len(D); st = dict(nodes=0, leaves=0, pruned=0, bound_evals=0)
    min_out = [min(D[i][j] for j in range(n) if j != i) for i in range(n)]              # preprocessing O(n^2)
    order = [sorted(range(1, n), key=lambda j, i=i: D[i][j]) if nearest_first else list(range(1, n)) for i in range(n)]
    best = [math.inf, None]
    if seed_ub:
        L, t = local_search(D, 5, rng or random.Random(0)); best = [L + 1e-9, t]       # incumbent from 2-opt (reused work)
    used = [False]*n; used[0] = True; path = [0]; rem = [sum(min_out[1:])]
    def extend(cost):
        st['nodes'] += 1
        if len(path) == n:
            st['leaves'] += 1; total = cost + D[path[-1]][0]
            if total < best[0]: best[0] = total; best[1] = path[:]
            return
        last = path[-1]
        for c in order[last]:
            if used[c]: continue
            c2 = cost + D[last][c]
            if use_lb:
                st['bound_evals'] += 1
                lb = c2 + (rem[0] - min_out[c]) + min_out[c]        # every unvisited city (and c itself) still needs one outgoing edge
                if lb >= best[0]: st['pruned'] += 1; continue
            elif c2 >= best[0]: st['pruned'] += 1; continue
            used[c] = True; path.append(c); rem[0] -= min_out[c]
            extend(c2)
            rem[0] += min_out[c]; path.pop(); used[c] = False
    extend(0.0)
    if best[1] is None: best[1] = best[1]
    return best[0], best[1], st
```

```
def clustered(n, seed):     rng = random.Random(seed); centres = [(rng.uniform(10, 90),
rng.uniform(10, 90)) for _ in range(3)]     return [(centres[i % 3][0] + rng.gauss(0, 4), centres[i %
3][1] + rng.gauss(0, 4)) for i in range(n)]

def twin_pairs(n, seed):
   """adversarial for the min-edge bound: cities come in very close twins, so every city's cheapest edge is
tiny,
   yet any tour must also make long hops between the pairs."""
   rng = random.Random(seed); pts =
[]     for _ in range(n//2):
       x, y = rng.uniform(0, 100), rng.uniform(0, 100); pts += [(x, y), (x+rng.uniform(0, 0.5),
y+rng.uniform(0,
0.5))]
return pts

def near_uniform(n, seed):
   """adversarial for bounds: all distances in [100, 101] -> min-edge bound is almost as large as any
edge"""
   rng = random.Random(seed); D = [[0]*n for _ in range(n)]
   for i in range(n):         for j in range(i+1, n): D[i][j] = D[j][i] =
rng.uniform(100, 101)     return D

VARIANTS = [("V0 original exhaustive", None), ("V1 prefix-cost pruning", dict(use_lb=False,
nearest_first=False, seed_ub=False)),
          ("V2 + lower bound", dict(use_lb=True, nearest_first=False, seed_ub=False)),
          ("V3 + nearest-first", dict(use_lb=True, nearest_first=True, seed_ub=False)),
("V4 + 2-opt seeded incumbent", dict(use_lb=True, nearest_first=True, seed_ub=True))]

if __name__ ==
"__main__":     out = {}
   print("PREDICTION (written before running): V0 expands ~ e*(n-1)! nodes. With pruning I expect
nodes to grow far more slowly "
         "(roughly c^n, c<n) on spread-out geometric instances; V4 (good incumbent from the start) should
be the cheapest; "
         "on near-uniform distances the bound is weak and the gain should almost vanish. Optimal cost must
be
identical for all variants.\n")
   print("A) Ablation, uniform random points, n=10 (10 instances): nodes expanded / tours completed /
bound
evals / time")
   print(f"{'variant':32s}{'nodes':>12s}{'leaves':>10s}{'bound evals':>13s}{'time
(ms)':>11s}{'same optimum':>14s}")     ab = []     for name, kw in VARIANTS:
       nodes = []; leaves = []; be = []; tm = []; same = True
for sd in range(10):
           D = dist_matrix(make_points(10, 50+sd)); opt = exhaustive(D)[0]
```

```
        t0 = time.perf_counter()
        if kw is None: c, _, k = exhaustive(D); nd = original_nodes(10); lv = k; b_ = 0
        else: c, _, st = bnb(D, rng=random.Random(sd), **kw); nd = st['nodes']; lv = st['leaves']; b_ =
st['bound_evals']
        tm.append((time.perf_counter()-t0)*1e3); same &= abs(c-opt) < 1e-6; nodes.append(nd);
leaves.append(lv); be.append(b_)
    ab.append((name, np.mean(nodes), np.mean(leaves), np.mean(be), np.mean(tm)))

print(f"{name:32s}{np.mean(nodes):12.0f}{np.mean(leaves):10.0f}{np.mean(be):13.0f}{np.mean(tm):1
1.1f}
{str(same):>14s}")
out['ablation'] = ab
  print("\nB) Scaling with n, uniform random points, 10 instances each (nodes expanded; V0 measured
for n<=11, formula beyond)")
  print(f"{'n':>3s}{'V0 original':>16s}{'V1':>12s}{'V2':>12s}{'V3':>12s}{'V4':>12s}{'V4 time
ms':>12s}{'speedup V0/V4':>16s}")    sc = []    for n in (6, 8, 10, 11, 12, 13, 14, 15):        row = [n,
original_nodes(n)]; tms = []        for vi, (name, kw) in enumerate(VARIANTS[1:]):            if (vi == 0
and n > 12) or (vi == 1 and n > 14):          # too slow to run: weaker variants skipped
            row.append(float('nan')); tms.append(float('nan')); continue
nds = []; tt = []            for sd in range(10):
            D = dist_matrix(make_points(n, 50+sd)); t0 = time.perf_counter(); c, _, st = bnb(D,
rng=random.Random(sd), **kw); tt.append((time.perf_counter()-t0)*1e3); nds.append(st['nodes'])
        row.append(np.mean(nds)); tms.append(np.mean(tt))
    row.append(tms[-1]); sc.append(row)

print(f"{n:3d}{row[1]:16d}{row[2]:12.0f}{row[3]:12.0f}{row[4]:12.0f}{row[5]:12.0f}{row[6]:12.1f}{r
ow[1]/ row[5]:16.1f}")    out['scaling'] = sc
  # verify formula for V0 at measured sizes
  print("\n  check of node formula for V0 at n=9: counted by instrumented run vs
formula")    D = dist_matrix(make_points(9, 1)); cnt = [0]    def count(path, used):
cnt[0] += 1
    if len(path) == 9: return        for c in range(1,
9):            if c not in used: count(path+[c], used |
{c})
  count([0], {0}); print(f"  counted = {cnt[0]}, formula = {original_nodes(9)}")
print("\nC) Instance structure (n=12, 10 instances each; V4 vs V0 formula)")
  print(f"{'instance type':28s}{'V0 nodes':>14s}{'V4 nodes':>12s}{'reduction x':>13s}{'V4 time
ms':>12s}")
  ins = []
  for label, gen in (("uniform random", lambda sd: dist_matrix(make_points(12, 70+sd))), ("clustered (3
clusters)", lambda sd: dist_matrix(clustered(12, 70+sd))),
                ("near-uniform distances", lambda sd: near_uniform(12, 70+sd)),
                ("twin pairs (weak bound)", lambda sd: dist_matrix(twin_pairs(12, 70+sd)))):
nds = []; tt = []        for sd in range(10):
```

```
        D = gen(sd); t0 = time.perf_counter(); c, _, st = bnb(D, rng=random.Random(sd));
tt.append((time.perf_counter()-t0)*1e3); nds.append(st['nodes'])
      ins.append((label, original_nodes(12), np.mean(nds), np.mean(tt))); print(f"{label:28s}
{original_nodes(12):14d}{np.mean(nds):12.0f}{original_nodes(12)/np.mean(nds):13.1f}{np.mean(tt):12.
1f}")
   out['instances'] = ins
   print("\nE) Small n: wall-clock V0 vs V4 (10 instances, ms) - where the original is still fine")
   print(f"{'n':>3s}{'V0 ms':>10s}{'V4
ms':>10s}")    small = []    for n in (4, 5, 6, 7,
8):
      a = []; b = []        for
sd in range(10):
        D = dist_matrix(make_points(n, 50+sd)); t0 = time.perf_counter(); exhaustive(D);
a.append(time.perf_counter()-t0)
        t0 = time.perf_counter(); bnb(D, rng=random.Random(sd)); b.append(time.perf_counter()-t0)
small.append((n, np.mean(a)*1e3, np.mean(b)*1e3)); print(f"{n:3d}{np.mean(a)*1e3:10.3f}
{np.mean(b)*1e3:10.3f}")
out['small'] = small
   print("\n  Near-uniform case at n=10, where V0 is cheap: V0 vs V4 wall-clock (V4 pays for bound
evaluation on every child)")
   D = near_uniform(10, 5); t0 = time.perf_counter(); exhaustive(D); t_v0 =
time.perf_counter()-t0    t0 = time.perf_counter(); c, _, st = bnb(D, rng=random.Random(0));
t_v4 = time.perf_counter()-t0    print(f"  V0: nodes={original_nodes(10)}, {t_v0*1e3:.1f} ms
|  V4: nodes={st['nodes']}, bound
evals={st['bound_evals']}, {t_v4*1e3:.1f} ms")
   out['near_uniform_n10'] = (original_nodes(10), t_v0, st['nodes'], st['bound_evals'], t_v4)
   # maximum n within 2 s
   print("\nD) Largest n solved to optimality within the 2 s budget by V4 (uniform random, 3 instances;
worst
instance reported)")
for n in (13, 14, 15):
      tt = []        for sd in
range(3):
        D = dist_matrix(make_points(n, 90+sd)); t0 = time.perf_counter(); bnb(D,
rng=random.Random(sd));
tt.append(time.perf_counter()-t0)
      print(f"  n={n}: mean {np.mean(tt):.2f} s, worst {max(tt):.2f} s")
json.dump(out, open("out/t7_raw.json", "w"), indent=1, default=float)
   fig, ax = plt.subplots(1, 2, figsize=(11, 4))
   ns = [r[0] for r in sc]
   ax[0].semilogy(ns, [r[1] for r in sc], 'k--o', label="V0 original ~(n-1)!")
   for i, lab in enumerate(["V1 prefix cost", "V2 + lower bound", "V3 + nearest-first", "V4 + seeded
incumbent"]): ax[0].semilogy(ns, [r[2+i] for r in sc], 'o-', label=lab)
   ax[0].set_xlabel("n"); ax[0].set_ylabel("nodes expanded (log)"); ax[0].legend(fontsize=7);
```

```
ax[0].set_title("Redesign ablation (uniform points)")
    ax[1].bar([i[0].replace(" ", "\n", 1) for i in ins], [i[1]/i[2] for i in ins], color=['#2a9d8f', '#e9c46a',
'#8ab17d', '#e76f51']); ax[1].set_yscale('log'); ax[1].set_ylabel("node reduction factor (V0/V4, n=12)");
ax[1].set_title("Gain depends on instance structure")
    plt.tight_layout(); plt.savefig("out/t7_charts.png", dpi=130); plt.close()
```